BEYOND TOKEN BOUNDARIES: BYTE-LEVEL PREDICTION FROM TOKENIZED LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenized language models condition on discrete encodings, but users supply text that may end inside a token. Conditioning on a single encoding can therefore disagree with conditioning on the observed bytes, and different model vocabularies complicate probability-level ensembling. We formulate byte prediction as marginalization over token histories that first cover the observed prefix, yielding an exact representation of the byte process induced by a token model without weight updates. We derive incremental inference and a reverse-search specialization, making explicit the support assumptions required for tokenizer-based pruning. On HumanEval Random Span, byte-level prediction raises CodeLlama-7b's suffix-prefix-middle greedy pass rate from 45.0 to 63.9 and its reported pass@10 from 66.5 to 84.3. Averaging predictions in byte space reaches 65.4 on MMLU and 42.1 pass@1 on HumanEval, exceeding the best reported individual scores of 63.4 and 38.4. Benefits are not universal: individual-model code generation can regress, and the GSM8K ensemble underperforms its strongest member. These results support byte-space conditioning for boundary-sensitive completion and vocabulary-independent ensembling, while leaving runtime and approximation error unresolved.

1 INTRODUCTION

Subword tokenization provides a practical vocabulary for language modeling, including representations of rare words (Sennrich et al., 2016). Yet a token boundary is an internal modeling choice, not a constraint on the text a user can provide. A code completion prompt may stop halfway through an identifier, and the tokenization of that partial identifier may differ from the corresponding prefix of the tokenization of the completed program. A model asked to continue the former token history can consequently make a different prediction from the one obtained by conditioning its underlying text distribution on the observed prefix.

We call this discrepancy *tokenization bias*. Here, bias denotes a difference between two conditioning operations on the same model, not social bias or necessarily a discrepancy from the true data distribution. The distinction matters: recovering the byte distribution induced by a model does not correct errors already present in that model. Nevertheless, the supplied synthetic experiments exhibit a particularly sharp boundary failure: a valid next byte receives zero probability under deterministic token-history conditioning even though its data-generating probability is positive.

Our approach retains the token model and changes inference. For an observed byte prefix, we consider all token histories whose final token first reaches or crosses its endpoint. These *cover encodings* partition the event of observing that prefix. Their probabilities give the prefix likelihood and, by taking ratios, the next-byte distribution. Crucially, a covering token may extend beyond the prompt: restricting the sum to tokenizations that end exactly at the prompt boundary would omit precisely the hypotheses needed for mid-token completion.

This formulation also resolves an interface problem in ensembling. Token probabilities from distinct vocabularies cannot be directly averaged by token index. Byte probabilities can: each model contributes a distribution over the same 256 byte values, together with a termination event. Figure 1 summarizes both applications.

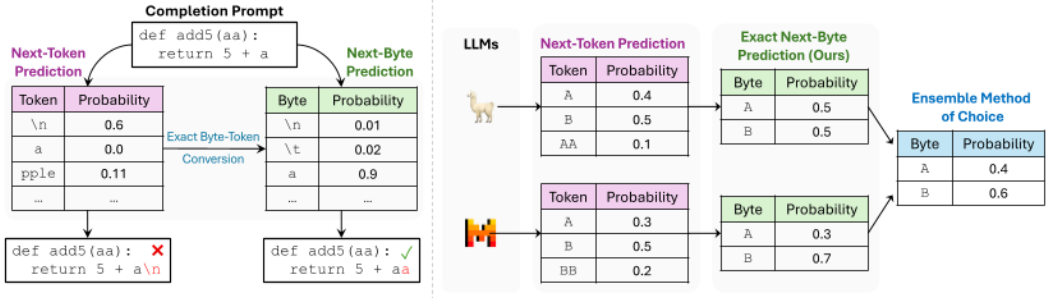


Figure 1: Boundary-sensitive completion and a common output space. Left: the supplied illustration contrasts token-history conditioning with byte-prefix conditioning when a prompt ends inside an identifier. Right: mapping different vocabularies into byte probabilities permits probability averaging. The displayed probabilities are illustrative, not benchmark measurements.

The paper makes three contributions. First, it gives a byte-token representation lemma and separates unconditional exactness of the representation from assumptions needed to prune token histories. Second, it specifies incremental marginalization and a reverse-search specialization that reuse token-model predictions without retraining. Third, it evaluates boundary-sensitive infilling, individual-model prediction, and heterogeneous ensembling using the recorded experiments. The strongest evidence concerns suffix-prefix-middle (SPM) infilling; mixed results elsewhere delimit the claim. We do not claim that byte-level decoding uniformly improves accuracy or that exact marginalization is inexpensive for every token model.

2 Related Work

Tokenization and segmentation. Subword units address vocabulary coverage in neural language processing (Sennrich et al., 2016). Tokenizer choices also affect pretraining and domain adaptation (Dagan et al., 2024). These studies motivate treating the tokenizer as part of the modeling interface rather than an interchangeable text preprocessing step. Marginalization over alternative segmentations has an established role in language modeling: Grave et al. (2019) trains hybrid models by summing over segmentations. Our focus is instead inference with a fixed pretrained token model, including histories whose last token overshoots an observed byte prefix. The representation identity is a probability decomposition; efficient evaluation requires additional structure in the model's support.

Infilling and boundary repair. Fill-in-the-middle training rearranges text to support generation conditioned on both a prefix and a suffix (Bavarian et al., 2022). Code Llama includes models with infilling capabilities (Rozière et al., 2023). Boundary repair is directly relevant when the supplied prefix ends inside a subword. Token healing backs up from the boundary and constrains regeneration to match the observed text; Dagan et al. (2024) discusses both the motivation and variants of this procedure. Token alignment uses character matching for subword completion (Athiwaratkun et al., 2024). Both provide relevant empirical baselines. Our mathematical target is the full model-induced byte conditional, which averages compatible histories according to their probability rather than selecting a single repaired history. The experimental records do not specify the exact healing or alignment implementation, so the comparisons apply to the recorded configurations.

Evaluation and heterogeneous models. HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) evaluate program synthesis. MMLU (Hendrycks et al., 2020), PIQA (Bisk et al., 2020), Natural Questions (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), and GSM8K (Cobbe et al., 2021) cover knowledge, physical reasoning, question answering, and mathematical problem solving. We use the results recorded for these benchmarks to examine whether boundary correction transfers beyond infilling. Our ensemble uses an arithmetic mean of byte conditionals, providing

a shared prediction space without requiring shared token identifiers. The comparison to voting is limited to the two benchmarks with recorded voting results.

3 METHODOLOGY

3.1 TOKEN PROCESSES, BYTE PREFIXES, AND CONDITIONING

Let $\mathcal{B} = \{0, \ldots, 255\}$ and let $\mathcal{V}$ be a finite vocabulary of ordinary tokens. Each token t has a nonempty byte string $d(t) \in \mathcal{B}^+$. Decoding concatenates these strings:

$$D(t_{1:k}) = d(t_1) \cdots d(t_k), \qquad P(t_{1:k}) = \prod_{i=1}^{k} P(t_i \mid t_{<i}). \tag{1}$$

Here $P(t_{1:k})$ is the probability of a *token prefix*, not of terminating after token k. Set $P(\varnothing) = 1$. The deterministic encoder E satisfies $D(E(x)) = x$ on representable strings. The derivation assumes concatenative, lossless byte decoding. A tokenizer with normalization or context-dependent decoding requires an explicit decoder state; the simple formulas below do not cover it automatically.

We condition on any fixed external control context, such as beginning-of-sequence or FIM control tokens, and suppress that context in the notation. Such controls must retain their model-specific semantics rather than being treated as ordinary text bytes. An end-of-sequence event `EOS` terminates the process and is handled separately. There are no other silent transitions in this formulation.

For a finite string x, define the byte-prefix mass

$$Q(x) = \Pr\{\text{the decoded token process begins with } x\}, \qquad Q(\varnothing) = 1. \tag{2}$$

Write $x \preceq y$ when x is a prefix of y. For $Q(x) > 0$, the desired next-byte probability is

$$p_{\text{byte}}(b \mid x) = \frac{Q(xb)}{Q(x)}, \qquad b \in \mathcal{B}. \tag{3}$$

In contrast, deterministic token-history conditioning gives

$$p_{\text{det}}(b \mid x) = \sum_{t \in \mathcal{V}:\, d(t)_1 = b} P(t \mid E(x)). \tag{4}$$

These condition on different events. A byte-level discrepancy can be measured by total variation between the two distributions, including their respective termination probabilities. The experiments do not report this quantity numerically.

3.2 THE BYTE-TOKEN REPRESENTATION LEMMA

For $n = |x| > 0$, define the set of minimal cover histories

$$\mathcal{C}(x) = \{h = t_{1:k} : \; x \preceq D(h), \quad |D(t_{1:k-1})| < n \leq |D(h)|\}. \tag{5}$$

Minimality means that the final token is the first to reach position n. It prevents double counting histories that merely append further tokens after the prefix is already covered. Set $\mathcal{C}(\varnothing) = \{\varnothing\}$.

Lemma 1 (Byte-token representation)**.** *For the token process defined above and every finite byte string x,*

$$Q(x) = \sum_{h \in \mathcal{C}(x)} P(h). \tag{6}$$

Consequently, whenever $Q(x) > 0$, equation 3 gives the exact next-byte conditional induced by that token process.

Proof. Every token realization whose decoding begins with x has a unique first token that reaches byte $|x|$. The token prefix through that token belongs to $\mathcal{C}(x)$. Conversely, every extension of a cover history begins with x. These token-prefix events are disjoint: extending a cover history cannot produce another minimal cover of the same x. Summing their probabilities proves the identity. □

The lemma does not assume that the model generates only canonical encodings. It is exact for the model distribution, regardless of whether that distribution matches the data. If the entire decoded process matches a data-generating byte process, its prefix conditionals also match wherever their denominator is positive. Recovering ground-truth transition probabilities in a synthetic example is therefore consistent with the lemma, but is not guaranteed for an imperfect learned model.

Termination must not be silently discarded. Define

$$Q_{\mathrm{stop}}(x) = \sum_{h:D(h)=x} P(h)P(\texttt{EOS} \mid h). \tag{7}$$

Then $Q(x) = Q_{\mathrm{stop}}(x) + \sum_{b\in\mathcal{B}} Q(xb)$. Sampling from the normalized joint distribution over bytes and termination preserves the token model's decoded process. Renormalizing only the byte entries instead conditions on producing at least one more byte. Appendix A details this distinction and a small symbolic example.

3.3 Valid encodings and reverse search

Call a token history *canonical* when $E(D(h)) = h$. A deterministic tokenizer may never emit a noncanonical sequence, but that fact alone does not imply $P(h) = 0$ for a learned token model. Discarding such a sequence changes the represented distribution if its probability is positive.

For an efficient specialization, impose the following explicit *canonical support condition*:

$$P(h) > 0 \quad \Longrightarrow \quad E(D(h)) = h \quad \text{for every finite ordinary-token history } h. \tag{8}$$

This is a sufficient assumption, not a property established by the experimental logs. Under it, a cover history whose final token starts at byte j must have the form $h = E(x_{1:j-1}) \parallel t$. Let $L = \max_{t\in\mathcal{V}} |d(t)|$. All candidates can be found by the following reverse search:

1. Visit $j = n, n-1, \ldots, \max(1, n-L+1)$.
2. Retrieve all tokens t for which $x_{j:n} \preceq d(t)$, using a vocabulary prefix index.
3. Construct $h = E(x_{1:j-1}) \parallel t$. Retain it if it passes the canonical check, and score it with equation 1.
4. Sum retained history probabilities to obtain $Q(x)$.

The empty preceding history is allowed. The search stops after L possible start positions because an earlier final token cannot reach byte n. Under equation 8, every positive-mass cover is considered and rejected histories have zero mass. Without this condition, exact search must retain every positive-mass preceding token history that decodes to $x_{1:j-1}$, rather than only its deterministic encoding.

Figure 2 illustrates the specialization. Its candidate count is at most $L|\mathcal{V}|$, before using prefix indexing. This is not a bound on end-to-end neural inference time: evaluating history probabilities and obtaining their next-token logits also costs computation. For unrestricted models, the number of compatible histories can grow exponentially with prefix length. Thus the exact representation for arbitrary token models should be distinguished from an efficient implementation under restricted support.

3.4 Incremental next-byte prediction

Maintain each $h \in \mathcal{C}(x)$ together with its mass $w_h = P(h)$ and its token-model state. Initialize from an exact cover enumeration, or from the reverse specialization when its assumption holds. Split the next step into two cases.

If $|D(h)| > |x|$, the token already fixes the next byte. Add w_h to the accumulator for byte $D(h)_{|x|+1}$ and keep h in the corresponding next cover set. If $|D(h)| = |x|$, query the next-token distribution at h. Each ordinary token t contributes $w_h P(t \mid h)$ to the accumulator for its first byte, and the extended history $h \parallel t$ becomes a cover of that extended byte prefix. Add $w_h P(\texttt{EOS} \mid h)$ to the termination accumulator. Summing across histories produces

$$A_b = Q(xb), \qquad A_{\mathrm{stop}} = Q_{\mathrm{stop}}(x), \qquad \sum_b A_b + A_{\mathrm{stop}} = Q(x). \tag{9}$$

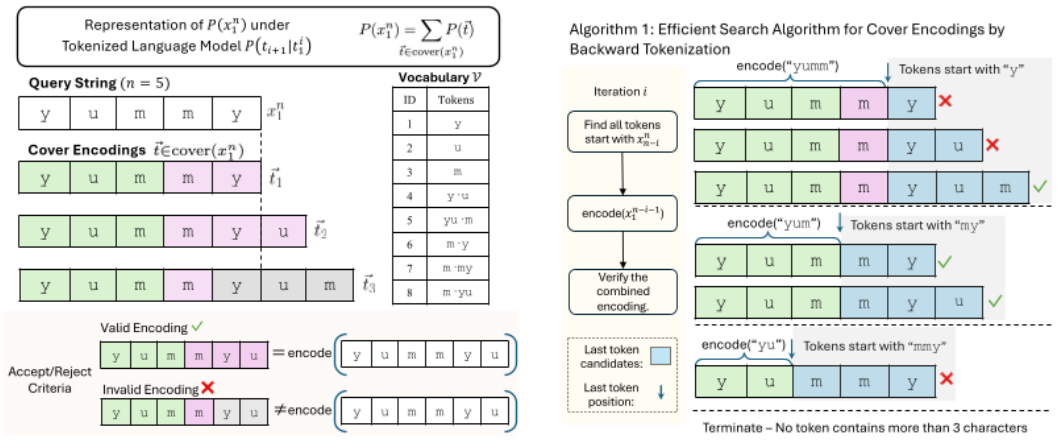


Figure 2: Cover histories and reverse search in the supplied BPE illustration. A covering token may extend past the observed string. The accept/reject checks shown in the artwork yield exact pruning only under the canonical support condition in equation 8; the general representation lemma does not require that condition.

Normalize by $Q(x)$, sample a byte or termination, and retain the associated cover set for the following step. Greedy byte decoding instead chooses the largest normalized entry. Unnormalized history masses can be maintained in log space with log-sum-exp aggregation. Caching model states avoids reprocessing an unchanged token history. Appendix C includes the supplied sampling illustration.

Under canonical support, at most one positive-mass history ends exactly at a particular byte prefix, namely $E(x)$. The incremental step then requires at most one new next-token distribution, while overshooting covers contribute already determined bytes. This does not imply one model call per generated token, or a measured speedup. Without canonical support, distinct boundary histories require distinct model states and predictions.

Byte-greedy and token-greedy decoding also optimize different local choices. Even at an unambiguous token boundary, the sum of probabilities of several tokens sharing a first byte can exceed the probability of the single most likely token. A gain from byte-greedy decoding therefore need not demonstrate removal of a mid-token error. Truncating histories or using a beam to approximate marginalization is not exact in general; the recorded beam experiment does not specify enough implementation detail to equate beam width with retained probability mass.

3.5 Ensembling in a common byte space

For M models, compute each model's byte conditional at the same observed text, using its own tokenizer and token probabilities. With $a \in \mathcal{B} \cup \{\text{stop}\}$, the recorded ensemble rule is

$$p_{\text{ens}}(a \mid x) = \frac{1}{M} \sum_{m=1}^{M} p_m(a \mid x). \tag{10}$$

This is a valid distribution when all members are defined and use consistent termination semantics. After selecting a byte, update each model's cover state separately. The rule is a stepwise arithmetic pool, not generally the posterior predictive distribution of a mixture that selects one model for an entire sequence. No improvement guarantee follows: an inaccurate member can reduce the probability assigned to a correct continuation. The experiments use two members; their selection and evaluation protocols are discussed below.

4 EXPERIMENTS

4.1 SETUP AND REPORTING SCOPE

The experimental record covers first- and third-order Markov chains, a six-hidden-layer GPT-2 architecture trained on synthetic data, and pretrained Transformer language models. General-purpose evaluations use Llama2-7B, Yi-1.5-6B, and Mistral-7B-v0.3. Infilling uses CodeLlama-7b. The code-generation table labels its models CodeLlama2-7b, CodeLlama2-13b, and Yi-Coder-1.5B; we preserve those labels because checkpoint identifiers are not supplied and the 7B naming differs across experiments.

Infilling is evaluated on HumanEval Random Span (Bavarian et al., 2022) in prefix-suffix-middle (PSM) and suffix-prefix-middle (SPM) modes. The baselines are ordinary token generation, healing, and alignment; the latter two are reported only for SPM. We reproduce greedy pass rates and the supplied @10 scores. The provided infilling figure specifies that @10 is the maximum over a temperature sweep, rather than a score at one fixed temperature. The grid, sampling seeds, and sample-count details are unavailable.

The general benchmarks are MMLU, PIQA, NQ, TriviaQA, and GSM8K; code generation uses HumanEval and MBPP. All reported scores are percentages. We retain the logs' accuracy and pass@1 terminology without assuming unreported answer extraction or normalization rules. Ensembles average byte probabilities of the top two models. The record does not establish how the top two were selected, whether selection used held-out data, or all task-specific pair identities. It explicitly identifies Yi-Coder-1.5B and CodeLlama2-13b for HumanEval, and discusses Yi-1.5-6B with Mistral-7B-v0.3 for GSM8K.

The records provide aggregate results, not example-level outputs, repeated runs, uncertainty intervals, compute measurements, or complete inference configurations. Accordingly, numerical differences below are descriptive; we make no statistical significance claim. Missing entries denote unreported measurements, not zero performance. Appendix E lists the principal reproducibility gaps.

4.2 SYNTHETIC VERIFICATION

The first-order example uses byte symbols A and B and a vocabulary containing A, B, and AA (Figure 3). The record reports a zero token-conditioned probability for A following context A, despite a byte transition probability $1 - \alpha$. A token history ending in standalone A can exclude futures that would instead have merged that byte into AA. Byte-prefix conditioning retains those histories through a cover token that straddles the observed boundary.

The third-order experiment reports severe predictive distortion under token conditioning and recovery of the chain's original conditional probabilities after conversion. The supplied plot (Appendix B) supports qualitative agreement between corrected and ground-truth predictions across the displayed contexts. The record does not include numeric errors, learned parameters, chain coefficients, training hyperparameters, or repeated runs. We therefore use these experiments as a qualitative mechanism check, not a quantified accuracy or calibration guarantee for arbitrary neural models.

4.3 CODE INFILLING AND BOUNDARY SENSITIVITY

Table 1 shows the clearest effect. Standard token-level greedy decoding drops from 64.9 in PSM to 45.0 in SPM, a 19.9 percentage-point decrease. Byte-level decoding reaches 63.9 in SPM, an 18.9-point gain over its token baseline; the corresponding PSM–SPM gap is only 1.8 points. The @10 SPM gain is 17.8 points, from 66.5 to 84.3. These are absolute percentage-point differences, not relative percentages.

Both boundary-repair baselines recover much of the SPM degradation. Byte-level decoding exceeds healing by 1.2 points greedily and 0.5 points at @10, and alignment by 0.9 and 1.4 points, respectively. The gains over these baselines are therefore much smaller than the gain over unmodified SPM decoding. In PSM, byte-level decoding improves greedy performance by 0.8 points and @10 by 0.1 points. These modest changes are compatible with the different local objectives of token-greedy and byte-greedy generation, in addition to any boundary effect.

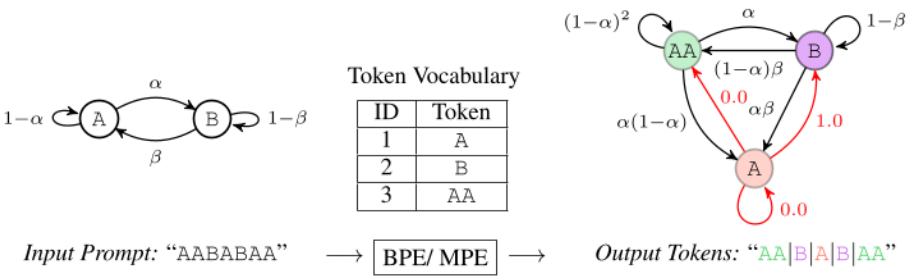


Figure 3: The supplied first-order Markov example. The byte chain and its token representation illustrate how conditioning on standalone token A excludes the next A, even when the byte transition has positive probability. The AA token provides a covering history that crosses the prompt boundary.

Table 1: HumanEval Random Span pass rates (%) with CodeLlama-7b. The @10 entries are reported maxima over a temperature sweep. Dashes indicate unreported settings. Bold marks the highest value in each column.

	PSM		SPM	
Method	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
Token-level + healing	—	—	62.7	83.8
Token-level + alignment	—	—	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

The qualitative record identifies partial identifiers such as `delimeter`, `collatz`, and `palindrome` as failure cases resolved by byte-level completion. These observations are consistent with the proposed mechanism, but neither their frequency nor a controlled isolation of boundary effects is available. The prompt-template comparison supports sensitivity to boundaries rather than proving that every PSM–SPM difference has the same cause. Appendix D reproduces the supplied infilling plot.

4.4 General prediction and heterogeneous ensembling

Byte-level prediction alone is mixed (Table 2). Several scores are unchanged or move by only a few tenths of a point. Llama2-7B, however, drops from 25.0 to 23.4 on NQ and from 15.1 to 12.5 on GSM8K. TriviaQA decreases for all three individual models. These results do not support treating byte conversion as a general capability improvement.

The ensemble exceeds the best reported individual result on four of five benchmarks: MMLU by 2.0 points (65.4 versus 63.4), PIQA by 0.4 (80.7 versus 80.3), NQ by 1.2 (30.0 versus 28.8), and TriviaQA by 0.6 (64.2 versus 63.6). Its improvements over the recorded voting baseline are 3.3 points on MMLU and 0.6 points on PIQA. Voting implementation and tie-breaking details are not reported, and voting results for the other tasks are unavailable.

GSM8K provides a clear counterexample: the ensemble's 55.8 is 5.7 points below Yi-1.5-6B's byte score of 61.5. The corresponding Mistral score is 39.0, a 22.5-point capability gap. Equal weighting can dilute the stronger model's predictions; this explanation is consistent with the result, but the aggregate scores alone do not establish a causal threshold for useful model diversity. The ensemble also uses multiple models, so the comparison is not matched for inference cost.

4.5 Code generation and beam width

Table 3 distinguishes individual conversion from ensembling. Every individual model loses HumanEval pass@1 after byte conversion: the changes are -3.6, -2.3, and -1.9 points. Nonetheless, the ensemble reaches 42.1, exceeding the best token-level individual score by 3.7 points and the best byte-level individual score by 5.6 points. Thus its gain is not evidence that conversion alone

Table 2: Knowledge and reasoning scores (%). T and B denote token- and byte-level prediction. Voting is the recorded top-two baseline; the ensemble averages byte probabilities. Bold marks the highest reported score per benchmark.

Model	Space	MMLU	PIQA	NQ	TriviaQA	GSM8K
Llama2-7B	T	45.4	78.1	25.0	58.4	15.1
	B	45.7	78.2	23.4	58.1	12.5
Yi-1.5-6B	T	63.4	78.5	22.8	53.7	61.3
	B	63.4	78.5	22.7	53.4	**61.5**
Mistral-7B-v0.3	T	62.1	80.1	28.5	63.6	39.0
	B	62.1	80.3	28.8	63.5	39.0
Voting (top two)	—	62.1	80.1	—	—	—
Top-two ensemble	B	**65.4**	**80.7**	**30.0**	**64.2**	55.8

Table 3: Code-generation pass@1 (%). Model labels follow the supplied table, including its CodeLlama2 naming. Bold marks the best score for each benchmark across both prediction spaces.

	HumanEval		MBPP	
Model	Token	Byte	Token	Byte
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-two ensemble	—	**42.1**	—	**53.6**

improves code synthesis. On MBPP, conversion helps the 7B and Yi-Coder entries but slightly hurts the 13B entry. The ensemble ties the best byte-level individual at 53.6 rather than exceeding it.

Table 4: Recorded byte-level beam-width ablation on HumanEval infilling. Scores are pass rates (%); the record does not specify beam scoring, length normalization, or whether beams contain output hypotheses or cover histories.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	67.7	66.8

Increasing the recorded beam width from one to four improves PSM by 2.0 points and SPM by 2.9 points (Table 4). The width-one entries match the greedy infilling results. The monotonic trend suggests that additional search helps these recorded configurations, but does not establish convergence to exact marginalization, a particular decoding objective, or a favorable accuracy–latency tradeoff. No beam runtime or token-level beam control is available.

5 Conclusion

Byte-prefix conditioning provides a principled interface between tokenized language models and arbitrary text boundaries. Minimal cover histories yield an exact representation of a token model's induced byte distribution; reverse search and incremental reuse make the computation structured, with efficient canonical pruning requiring a separate support assumption. The experimental evidence is strongest for SPM code infilling, where byte-level prediction recovers most of the degradation relative to PSM. A shared byte space also enables ensembles that improve several recorded benchmarks.

The results establish neither universal accuracy gains nor inexpensive exact inference for unrestricted neural models. Individual-model regressions, the GSM8K ensemble loss, missing uncertainty estimates, and unmeasured pruning error limit stronger claims. Future work should measure

noncanonical probability mass and inference cost, specify reproducible decoding protocols, and evaluate ensemble weighting using held-out selection. The essential benefit is a coherent conditioning and aggregation space; its computational and task-level value must be assessed separately.

REPRODUCIBILITY STATEMENT

The paper provides the probability identity, its proof, explicit support and decoder assumptions, a reverse-search procedure, and an incremental update rule. All tabulated measurements reproduce the supplied experimental record; additional numbers in the analysis are arithmetic differences of those measurements. The supplied figures are retained as illustrations or qualitative evidence. Exact experimental replication is currently limited by missing checkpoint revisions, code, dataset configurations, prompts, seeds, and inference settings, detailed in Appendix E. In particular, the records do not establish whether the evaluated wrapper preserves all noncanonical probability mass. No unavailable implementation detail is treated as a measured fact.

REFERENCES

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Robert Kwiatkowski, Ramesh Nallapati, Parminder Bhatia, and Bing Xiang. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 15725–15738, 2024. doi: 10.18653/v1/2024.findings-acl.929.

Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. doi: 10.48550/arXiv.2108.07732.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark I-Cheng Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022. doi: 10.48550/arXiv.2207.14255.

Yonatan Bisk, Rowan Zellers, Ronan Joseph Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239.

Mark I-Cheng Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024. doi: 10.48550/arXiv.2402.01035.

Édouard Grave, Sainbayar Sukhbaatar, Piotr Bojanowski, and Armand Joulin. Training hybrid language models by marginalizing over segmentations. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1477–1482, 2019. doi: 10.18653/v1/P19-1143.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. doi: 10.48550/arXiv.2009.03300.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pp. 1601–1611, 2017. doi: 10.18653/v1/P17-1147.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019. doi: 10.1162/tacl_a_00276.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023. doi: 10.48550/arXiv.2308.12950.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pp. 1715–1725, 2016. doi: 10.18653/v1/P16-1162.

A PROBABILITY IDENTITIES AND EXACTNESS

Finiteness and disjointness. For $n > 0$, each token in a cover history contributes at least one byte, so its token count is at most n. With finite $\mathcal{V}$, the set in equation 5 is finite. Its cardinality can nevertheless grow exponentially in n. If one cover history were a strict token-prefix extension of another, its penultimate decoding would already contain n bytes, contradicting minimality. Thus the corresponding token-cylinder events are pairwise disjoint. This establishes the sum without assuming a unique tokenization of any byte string.

Termination and normalization. Conditional on a decoded prefix x, a realization either stops exactly at x or supplies one next byte b. Stopping exactly at x requires an ordinary-token history h with $D(h) = x$ followed by `EOS`, yielding equation 7. All other realizations fall into exactly one event with prefix xb. Consequently,

$$p_{\mathrm{stop}}(x) + \sum_{b \in \mathcal{B}} p_{\mathrm{byte}}(b \mid x) = 1, \qquad p_{\mathrm{stop}}(x) = \frac{Q_{\mathrm{stop}}(x)}{Q(x)}. \tag{11}$$

If a task forces continuation, the correct conditional is $Q(xb)/(Q(x) - Q_{\mathrm{stop}}(x))$, provided the denominator is positive. This is a different conditioning event from the unmodified token process. The experimental logs do not specify their end-of-generation handling, so that choice cannot be reconstructed from the reported scores.

A symbolic boundary example. Consider a vocabulary with byte strings A, B, and AA and a prompt consisting of the single byte A. Denote token probabilities at the initial context by $P(A)$ and $P(AA)$. Its two covers are the one-token histories (A) and (AA), so

$$Q(A) = P(A) + P(AA), \tag{12}$$

$$Q(AA) = P(AA) + P(A)\big[P(A \mid A) + P(AA \mid A)\big]. \tag{13}$$

The first term in $Q(AA)$ comes from a token already spanning both bytes. Deterministic conditioning on token A omits it. If tokens beginning in A have zero conditional probability after standalone token A, but $P(AA) > 0$, then $p_{\mathrm{det}}(A \mid A) = 0$ while $p_{\mathrm{byte}}(A \mid A) = P(AA)/(P(A)+P(AA)) > 0$. This is a logical illustration of the failure mode, not an additional experiment or a fitted parameterization of the supplied Markov chain. An exact zero requires the stated support restriction; the records do not establish literal zeros for every pretrained neural model.

Completeness of reverse search under canonical support. Let $h = (u, t)$ be any positive-mass cover of x, with $j = |D(u)|+1$. Its final token has length at most L, so $j \geq n-L+1$. Since h covers x, $D(u) = x_{1:j-1}$ and $x_{j:n} \preceq d(t)$. Positive mass of h implies positive mass of u. Under equation 8, $u = E(x_{1:j-1})$ and $h = E(D(h))$. The reverse search therefore constructs and accepts h. Every accepted candidate is a minimal cover by construction. Any omitted noncanonical cover has zero probability under the assumption. Without that assumption, rejection based only on deterministic re-encoding is not justified.

Exactness versus approximation. The general algorithm must distinguish token histories even when they decode to the same bytes: their next-token probabilities need not agree. A cache may share computation for identical token prefixes, but cannot collapse different histories merely because their decoded strings match. If a cover approximation retains a subset $R(x)$ with discarded conditional mass

$$\delta(x) = 1 - \frac{\sum_{h \in R(x)} P(h)}{Q(x)}, \tag{14}$$

then renormalizing the retained histories changes the next-event distribution by at most $\delta(x)$ in total variation, *provided* each retained history's next-event distribution is evaluated exactly. This follows by writing the full conditional as a mixture of retained and discarded-history conditionals. No discarded-mass values are available for the reported experiments; a beam width alone does not bound $\delta(x)$.

B THIRD-ORDER SYNTHETIC VERIFICATION

Figure 4 shows the supplied third-order experiment at a scale that preserves its context labels. The horizontal axis is the probability that the next byte is A, and the vertical axis lists the last three bytes. The corrected bars qualitatively agree with the ground-truth bars. The plot does not supply numerical error values or uncertainty estimates, and we do not infer them from pixel positions.

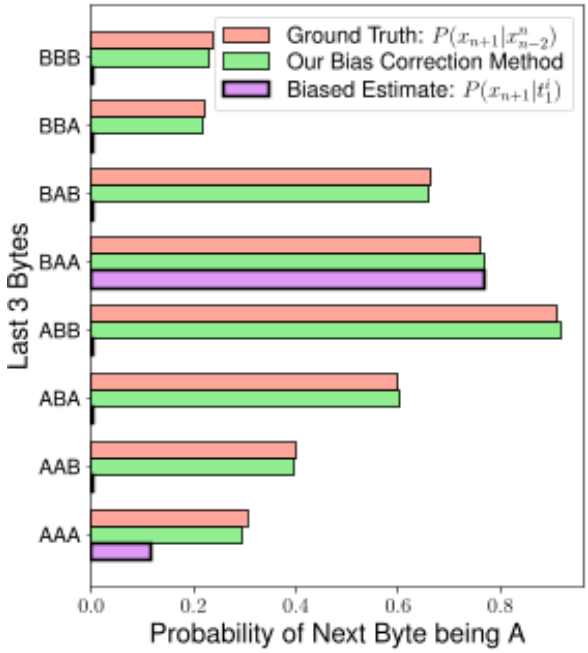


Figure 4: Supplied third-order Markov-chain comparison of ground truth, corrected byte predictions, and token-conditioned estimates. This is qualitative evidence of transition recovery for the displayed contexts.

C INCREMENTAL SAMPLING ILLUSTRATION

Figure 5 expands the two update cases in Section 3.4: carry forward histories whose final token already extends beyond the current prefix, and extend histories ending exactly at the prefix with next-token probabilities. The figure depicts the canonical specialization, including accept/reject checks and a single boundary-history model call. The general algorithm must keep all positive-mass boundary histories and must not perform unsupported rejection. Each history's prefix probability is essential even when omitted from the artwork for clarity.

D SUPPLIED INFILLING VISUALIZATION

Figure 6 reproduces the available infilling visualization. Its table agrees with Table 1, and its annotation identifies the @10 scores as maxima over the plotted temperature sweep. The raw values underlying the curves are not supplied. We therefore do not extract additional performance estimates from their geometry or assume a particular temperature grid.

E EXPERIMENTAL REPORTING DETAILS AND LIMITATIONS

The available evidence consists of aggregate tables, qualitative observations, and the six supplied figures. The following missing details constrain replication and interpretation.

- **Models and tokenizers.** Checkpoint revisions, tokenizer versions, precision, context limits, and special-token handling are unspecified. The setup calls the infilling model

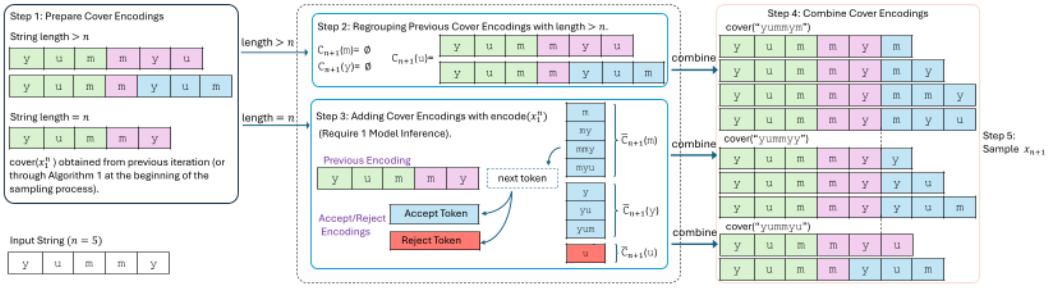


Figure 5: Supplied sampling illustration. Overshooting cover tokens determine their next byte directly, while histories ending at the current prefix require next-token probabilities. Actual aggregation multiplies conditional token probabilities by the probability of the preceding history. The depicted rejection rule requires the support qualification discussed in the main text.

Predictions	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

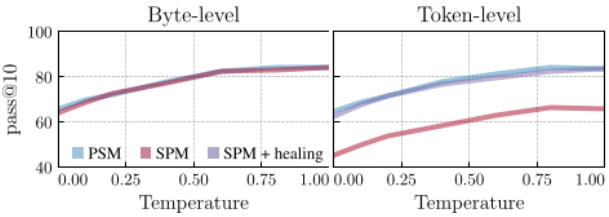


Figure 6: Supplied HumanEval Random Span visualization for CodeLlama-7b, including the reported table and temperature curves. The figure supports the qualitative comparison of PSM, SPM, and healing; Table 1 is the numerical reporting source.

CodeLlama-7b, while the code-generation table uses CodeLlama2-7b and CodeLlama2-13b. These names are preserved without inferring checkpoint equivalence.

- **Synthetic experiments.** Transition matrices, values of α and β, training-set sizes, optimizer settings, random seeds, and numerical probability-recovery errors are not recorded. The six-layer GPT-2 architecture is mentioned without a complete mapping from training runs to plotted results.
- **Evaluation.** Dataset splits and revisions, exact prompts, few-shot counts, span-sampling rules, generation budgets, answer extraction, test-execution settings, and pass-rate estimators are unspecified. The @10 temperature-maximization annotation is available, but the search grid and selection procedure are not.
- **Inference.** The record does not specify whether noncanonical histories are retained, how probability mass is truncated, how termination is treated, or how beam scores and stopping rules are defined. Consequently, the mathematical exactness theorem cannot certify the numerical implementation from the logs alone.
- **Ensembles and voting.** The record describes top-two selection and equal probability averaging but does not give a complete task-by-task member list, held-out selection protocol, or voting tie-breaking rule. The identified HumanEval pair is Yi-Coder-1.5B with CodeLlama2-13b; the GSM8K discussion identifies Yi-1.5-6B with Mistral-7B-v0.3.
- **Uncertainty and cost.** No per-example predictions, repeated-run dispersion, confidence intervals, hardware descriptions, memory measurements, throughput, or latency are available. Small score differences and compute efficiency therefore remain unverified beyond the reported aggregates.

These omissions do not change the values reproduced in the tables, but they prevent a complete independent replication or attribution of every observed difference to exact byte marginalization.